ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA

SOFTWARE ARCHITECTURE
FINAL PROJECT REPORT
QUICKSHIP – LOGISTIC MANAGEMENT PLATFORM

Instructor: Vũ Quang Dũng

Course: Software Architecture – Class N02

Team Members: Group 11

Nguyễn Trọng Thái – 23010457

Nguyễn Minh Quang – 23010419

Hà Nội, tháng 1 năm 2026

JOB ROSTER

No.	Member Name	Assigned Tasks	Completion Evaluation	Contribution (%)
1	Nguyễn Trọng Thái	- Project lead and document coordinator. - Compiled and formatted full report (SRS, Architecture, Database Design, Testing). - Designed system architecture diagrams (Layered, C4 Model). - Implemented UI components: AdminPage, DriverPage, CODManagementPage. - Performed system testing and verification documentation.	Completed successfully	50%
2	Nguyễn Minh Quang	- Collected and analyzed system requirements. - Designed use case, activity, and data flow diagrams. - Developed core logic: AuthContext, LanguageContext, and Routing system.- Implemented Customer-side pages (CreateOrder, TrackOrder, Promotions). - Supported preparing presentation and proofreading.	Completed successfully	50%

Contents

1. Executive Summary

1.1 Project Objectives & Vision

Quickship is an integrated internal logistics management ecosystem designed to streamline the end-to-end delivery process. The project aims to bridge the communication gap between dispatchers, field operators, and end-receivers by centralizing data into a single source of truth. The system caters to three primary stakeholder groups:

- **Admin (Operations Manager):** Acts as the command center, responsible for order entry, strategic driver dispatching, and comprehensive fleet oversight.
- **Driver (Field Personnel):** Executes deliveries with real-time progress updates, ensuring transparency from the warehouse to the customer's doorstep.
- **Customer (End User):** Empowers users with self-service tracking capabilities, providing transparency for shipment milestones and COD (Cash on Delivery) verification.

1.2 Technical Architecture

The system utilizes a **Layered Monolithic Architecture**. This structural choice ensures a clean **Separation of Concerns (SoC)**, making the codebase highly maintainable and preparing the system for a seamless transition to a full-stack production environment.

Architecture Layer	Technology Stack	Functional Role
Presentation	React, Tailwind CSS	Delivers a responsive, mobile-first UI with high-performance rendering and a modern aesthetic.
Application	Context API, Custom Hooks	Manages complex business logic, global state transitions, and role-based access control (RBAC).
Data	LocalStorage, Mock API	Simulates persistence and data retrieval, allowing for a fully functional prototype without immediate backend dependencies.
Infrastructure	Vite, Node.js, Vercel	Provides a modern development toolchain and cloud-native deployment for high availability.

1.3 Key Achievements & Value Proposition

The development of Quickship has resulted in a robust "Proof of Concept" that mirrors enterprise-grade logistics software:

- **Full CRUD Lifecycle:** Successfully implemented a comprehensive system to Create, Read, Update, and Delete data across orders, users, and logistics assets.
- **Standardized Workflow:** Established a rigorous **Order Lifecycle**—tracking shipments through various stages: *Pending, Dispatched, In Transit, Delivered,* or *Returned.*
- **Extensibility:** The modular frontend architecture is designed to be "plug-and-play," allowing developers to swap the Mock API for a real RESTful or GraphQL backend with minimal refactoring.
- **Operational Efficiency:** By digitizing manual logs, the system reduces human error in COD collection and improves the speed of driver communication.

Key Takeaway: Quickship is more than a simulation; it is a scalable framework designed to handle the complexities of modern "Last-Mile" delivery logistics.

2. Project Requirements & Goals

1. System overview description

1.1. Operating environment

The system works on modern browsers (Chrome, Edge, Firefox).
Connect to **Mock API/LocalStorage** to save simulation data.
Ready to integrate with **real databases (PostgreSQL/MySQL)** via backend API.

1.2 User characteristics

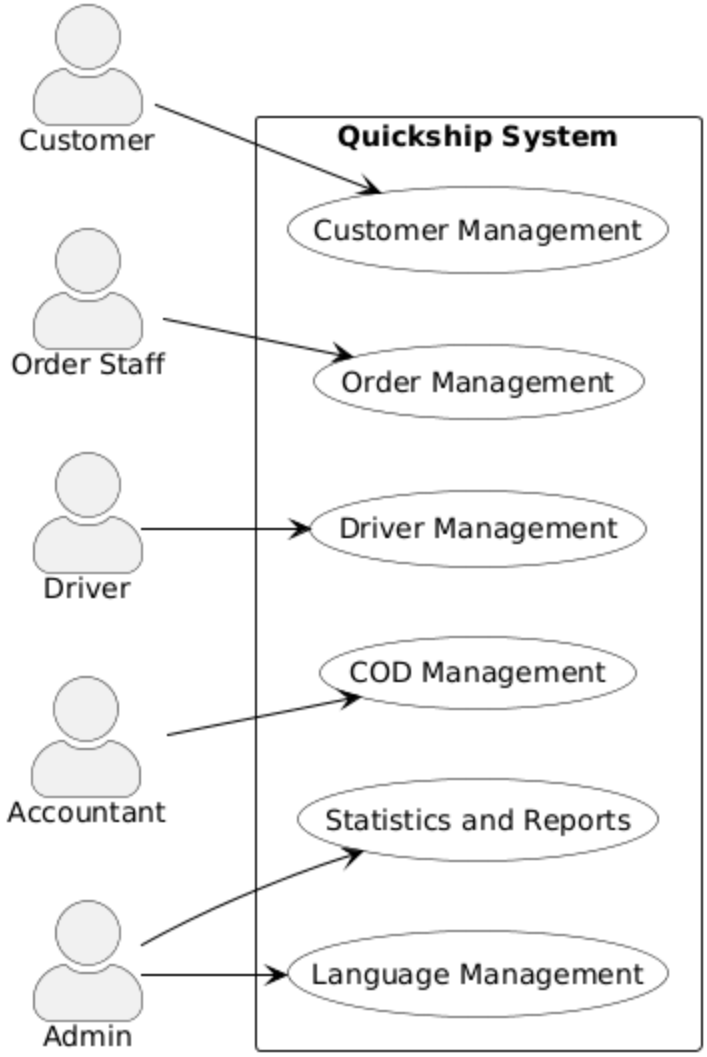
User Type	Description	Main tasks
Customer	Sender/Orderer	Create, track, view COD
Driver	Delivery person	Receive orders, update status
Admin	System Operator	Order management, drivers, statistics

1.3 Limitations and Constraints

- No constant network connection required (for using Mock Data).
- Online payments are not currently supported.
- Simple security: only login simulation (no JWT encryption yet).
- All data is reinitialized when resetting the browser.

2. Functional Requirements

2.1. Functional hierarchy diagram

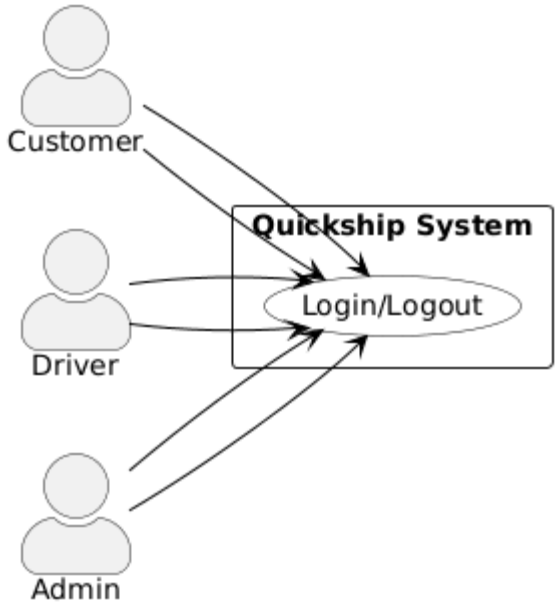


2.2. Detailed Function List

2.2.1. Login/Authentication Function

Item	Description
Function Code	F01
Use Case Name	Login/Logout
Actor	Customer, Driver, Admin
Description	The user enters an email, password, and role to access the system.
Precondition	The user has a valid account in the system.
Postcondition	The user is successfully logged in and redirected to the corresponding interface.
Input	- Email- Password- Role
Output	- Login status (success/failure)- Redirected interface (Customer, Driver, or Admin)
Main Flow	1. User opens the login page. 2. User enters email, password, and selects role.

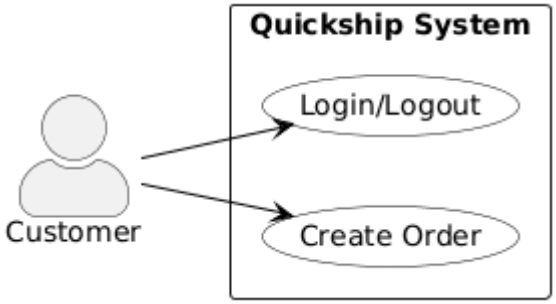
Item	Description
	3. System verifies the credentials. 4. If valid, system grants access and redirects user to the corresponding dashboard. 5. If invalid, system displays an error message.
Alternative Flow	- **Invalid Credentials:** System displays "Invalid email or password."- **Account Locked:** System displays "Account temporarily locked."
Go to	Customer interface, Driver interface, Admin interface



2.2.2. Order Creation Function

Item	Description
Function Code	F02
Use Case Name	Create Order
Actor	Customer
Description	Allows customers to create a new shipping order that includes sender, recipient, address, item type, and COD information.
Precondition	The customer is logged into the system.
Postcondition	A new shipping order is successfully created and assigned a unique bill of lading code.
Input	Order information: sender, receiver, address, item type, COD
Output	- Bill of lading code- Confirmation or success message

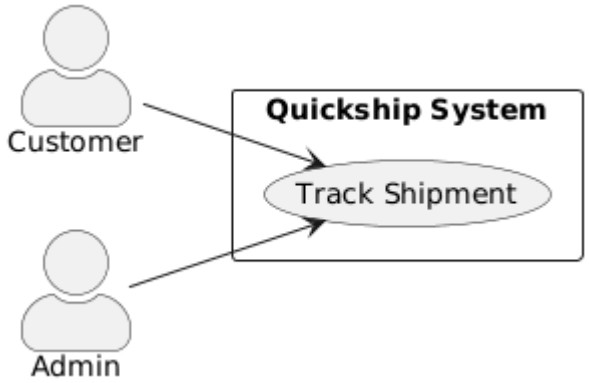
Item	Description
Main Flow	1. Customer selects "Create Order." 2. System displays the order form. 3. Customer enters sender, recipient, address, item type, and COD details. 4. System validates the entered data. 5. System generates a bill of lading code. 6. System confirms successful order creation.
Alternative Flow	- **Missing Required Fields:** System displays an error message.- **Invalid Data:** System prompts user to correct the information.
Go to	Customer Interface (Order List / Order Tracking)



2.2.3. Order tracking functionality

Field	Description
Function ID	F03
Use Case Name	Track Shipment
Actor	Customer, Admin
Description	Users can check delivery progress using a tracking code or sender's phone number.
Precondition	The shipment exists in the system.
Postcondition	The system displays the current shipment status to the user.
Input	Tracking code or sender phone number
Output	Current shipment status: "Delivering", "Delivered", or "Pending pickup"
Main Flow	1. User opens the tracking feature. 2. User enters tracking code or sender's phone number. 3. System searches for matching shipment(s).

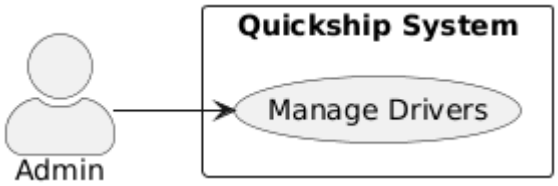
Field	Description
	4. System retrieves and displays current status.
Alternative Flow	- **Invalid Tracking Code:** System displays "Shipment not found."- **Network Error:** System prompts to retry.
User Roles	Customer, Admin



2.2.4. Driver management function

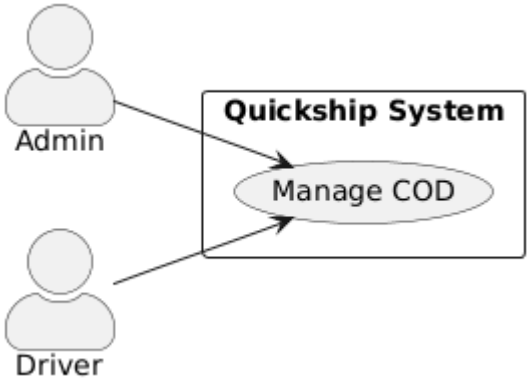
Field	Description
Function ID	F04
Use Case Name	Manage Drivers
Actor	Admin
Description	Admin can add, edit, or delete driver records, assign deliveries, and monitor driver performance.
Precondition	Admin is logged into the system.
Postcondition	Driver records are updated, and performance data is available for review.
Input	Driver information: name, phone number, status, number of deliveries
Output	- Driver list- Performance report
Main Flow	1. Admin selects "Manage Drivers." 2. System displays the list of drivers. 3. Admin can add, edit, or delete driver information. 4. Admin can assign deliveries to drivers. 5. System updates and saves driver data.6. System generates driver performance reports.
Alternative	- **Invalid Input:** System prompts admin to correct

Field	Description
Flow	the information.- **Driver Not Found:** System displays “No driver found.”
User Role	Admin



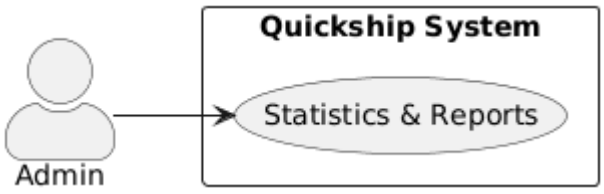
2.2.5. COD management function

Field	Description
Function ID	F05
Use Case Name	Manage COD (Cash on Delivery)
Actor	Admin, Driver
Description	Track, summarize, and verify COD payments per order or per driver.
Precondition	Orders with COD values exist in the system.
Postcondition	COD payments are verified, summarized, and recorded accurately.
Input	- Order ID- COD value
Output	- COD summary table- Total amounts grouped by driver or region
Main Flow	1. User (Admin/Driver) selects “Manage COD.” 2. System retrieves all orders with COD data. 3. User filters or searches by Order ID or driver. 4. System displays COD summary table. 5. System calculates totals by driver or region. 6. Admin verifies and updates payment status.
Alternative Flow	- **Order Not Found:** System displays “No matching order.”- **Data Mismatch:** System prompts to recheck COD value.
User Roles	Admin, Driver



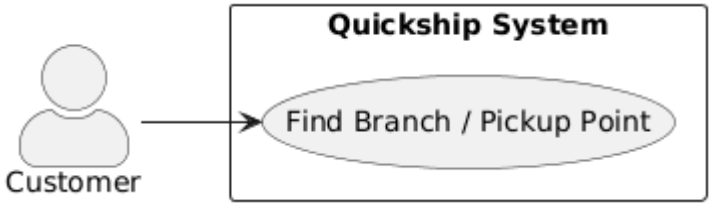
2.2.6. Statistics & Reporting Functions

Field	Description
Function ID	F06
Use Case Name	Statistics & Reports
Actor	Admin
Description	Display charts showing shipment volume, revenue, and delivery efficiency by day, week, or month.
Precondition	The system contains recorded shipment and financial data.
Postcondition	Statistical charts and reports are generated and available for download.
Input	- Time range (day/week/month)- Report type (shipment volume, revenue, efficiency)
Output	- Charts- Growth percentages- Downloadable PDF/Excel reports
Main Flow	1. Admin selects "Statistics & Reports." 2. System displays available report types and time range options. 3. Admin chooses parameters. 4. System retrieves relevant data. 5. System generates charts and growth percentages.6. Admin can download report as PDF or Excel.
Alternative Flow	- **No Data Available:** System displays "No data found for selected period."- **Export Error:** System notifies user to retry export.
User Role	Admin



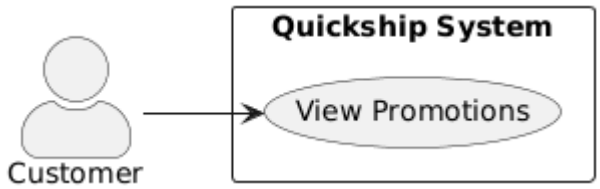
2.2.7. Shipping point search function

Field	Description
Function ID	F07
Use Case Name	Find Branch / Pickup Point
Actor	Customer
Description	Allows users to find Quickship offices or collection points on the map.
Precondition	The system has access to branch location data and Google Maps API.
Postcondition	The system displays nearby Quickship branches or pickup points based on the user's input.
Input	- Area name- GPS coordinates
Output	- List of nearby locations- Locations displayed on Google Maps
Main Flow	1. Customer selects "Find Branch / Pickup Point." 2. System displays a search box and map. 3. Customer enters area name or allows GPS access. 4. System retrieves branch/pickup point data. 5. System displays results as markers on Google Maps.
Alternative Flow	- **Invalid Location:** System displays "No branch found in this area."- **GPS Access Denied:** System requests manual input.
User Role	Customer



2.2.8. Promotions & Offers Function

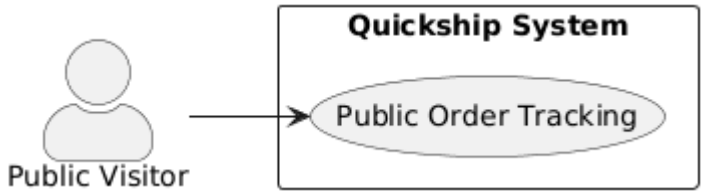
Field	Description
Function ID	F08
Use Case Name	View Promotions
Actor	Customer
Description	Display current discount codes and promotional campaigns available to customers.
Precondition	The system has valid and active promotion data.
Postcondition	Customer can view and apply available promotions or discount codes.
Input	*(None required)*
Output	- List of active promotions- Discount codes and details
Main Flow	1. Customer selects "View Promotions." 2. System retrieves all active promotions from the database. 3. System displays promotions and discount codes. 4. Customer can view details and copy discount code for use when creating orders.
Alternative Flow	- **No Active Promotions:** System displays "No promotions available."
User Role	Customer



2.2.9. Bill of lading lookup function (public)

Field	Description
Function ID	F09
Use Case Name	Public Order Tracking
Actor	Public Visitor
Description	Visitors can check order status without logging in.
Precondition	The system must have order information available for public access.
Postcondition	The system displays order details and current delivery status.
Input	Tracking code

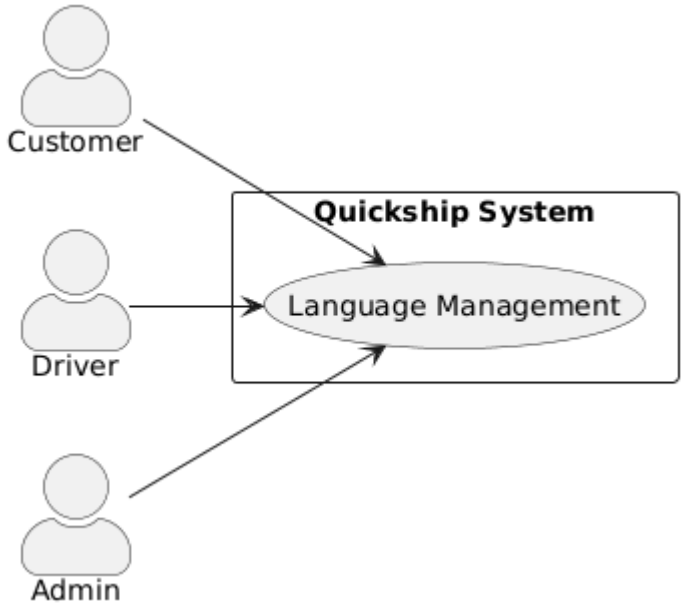
Field	Description
Output	- Order details- Current delivery status
Main Flow	1. Public visitor opens the "Public Order Tracking" page. 2. Visitor enters the tracking code. 3. System searches for the corresponding order. 4. System displays order details and delivery status (e.g., "Pending pickup," "Delivering," "Delivered").
Alternative Flow	- **Invalid Tracking Code:** System shows "Order not found."- **Expired Tracking Code:** System displays "Tracking unavailable."
User Role	Public Visitor



2.2.10. Language management function

Field	Description
Function ID	F10
Use Case Name	Language Management
Actors	Customer, Driver, Admin
Description	Supports bilingual interface switching between English and Vietnamese.
Precondition	The system interface supports multiple languages.
Postcondition	User interface is displayed in the selected language.
Input	*(No input required — user selects language option)*
Output	- UI displayed in the chosen language (English or Vietnamese)
Main Flow	1. User selects "Language Management" option. 2. System displays available language options (English, Vietnamese). 3. User selects preferred language. 4. System updates and reloads the interface in the chosen language.
Alternative	- **Unsupported Language:** System defaults to

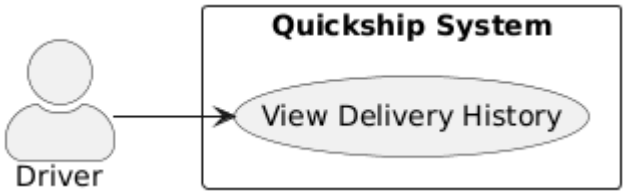
Field	Description
Flow	English.- **Session Timeout:** System reverts to default language on next login.
User Roles	Customer, Driver, Admin



2.2.11. Chức năng xem lịch sử giao hàng (Driver)

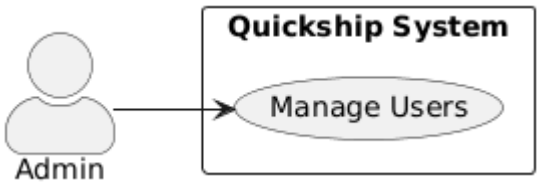
Field	Description
Function ID	F11
Use Case Name	View Delivery History
Actor	Driver
Description	Allows drivers to view a list of completed deliveries, including delivery time, destination, and status.
Precondition	The driver is logged in and has completed at least one delivery.
Postcondition	The system displays the driver's delivery history records.
Input	*(No manual input required — system filters by logged-in driver)*
Output	- List of past deliveries- Delivery details (order ID, delivery time, destination, COD amount, status)
Main Flow	1. Driver selects "View Delivery History." 2. System retrieves all completed delivery records for that driver. 3. System displays a list of deliveries with details (order ID, date, time, and status). 4. Driver can filter or sort by date or destination.

Field	Description
Alternative Flow	- **No Delivery Found:** System displays "No completed deliveries."- **Connection Error:** System shows "Unable to load data."
User Role	Driver



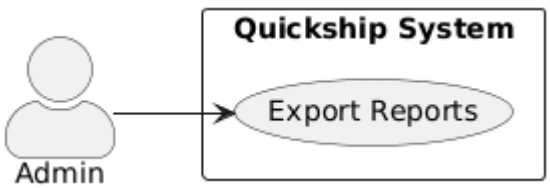
2.2.12. User Management Function (Admin)

Field	Description
Function ID	F12
Use Case Name	Manage Users
Actor	Admin
Description	Admin can view all registered users, assign roles, and lock or unlock user accounts.
Precondition	Admin is logged in with system management privileges.
Postcondition	User list and roles are updated successfully in the system.
Input	- User information (name, email, account status)- Role assignment (Customer, Driver, Admin)
Output	- Updated user list- Confirmation of role or status changes
Main Flow	1. Admin selects "Manage Users." 2. System displays all users in the system. 3. Admin can search, filter, and select a user. 4. Admin assigns or modifies user roles. 5. Admin can lock/unlock accounts. 6. System saves changes and confirms update.
Alternative Flow	- **Invalid User Data:** System displays "User not found."- **Permission Denied:** System shows "You do not have permission to modify this account."
User Role	Admin



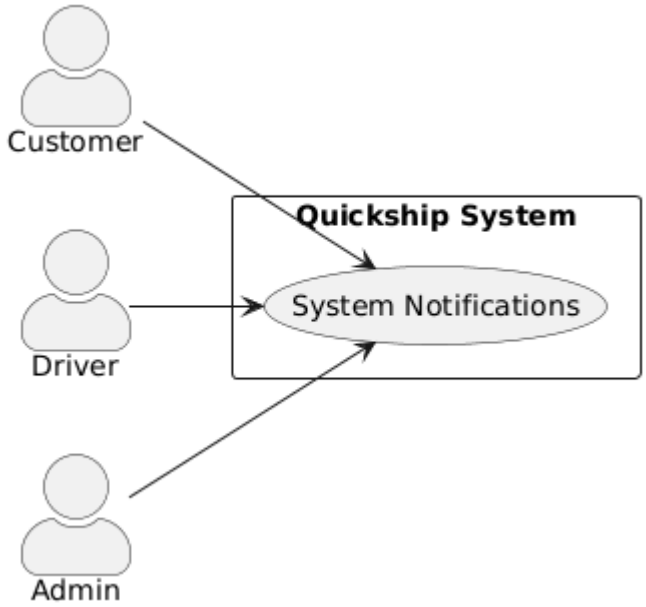
2.2.13. Report export function

Field	Description
Function ID	F13
Use Case Name	Export Reports
Actor	Admin
Description	Allows Admin to export system data (orders, drivers, COD) to PDF or Excel format for record keeping and analysis.
Precondition	Admin is logged in and has access to reporting data.
Postcondition	The selected report is exported successfully to a .pdf or .xlsx file.
Input	- Time range- Report type (Orders, Drivers, COD)
Output	- Exported file in .pdf or .xlsx format
Main Flow	1. Admin selects "Export Reports." 2. System displays report type and time range options. 3. Admin chooses parameters and export format (PDF/Excel). 4. System generates report data. 5. System exports file and notifies Admin upon completion.
Alternative Flow	- **No Data Found:** System displays "No data available for selected period."- **Export Error:** System shows "Failed to generate file."
User Role	Admin



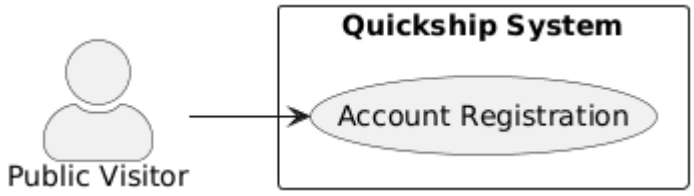
2.2.14. Notification Management Function

Field	Description
Function ID	F14
Use Case Name	System Notifications
Actors	Customer, Driver, Admin
Description	Sends updates or promotional messages to users when order status changes or new campaigns are available.
Precondition	User is logged in and has an active system session.
Postcondition	The user receives notifications via popup or banner message.
Input	- Notification content- Notification type (system update, order status, promotion)
Output	- Popup or banner message displayed to the user
Main Flow	1. System detects an event (order status change, new promotion, or admin update). 2. System creates a notification message with appropriate content and type. 3. Notification is pushed to users (Customer, Driver, Admin). 4. User views notification on dashboard or via popup/banner.
Alternative Flow	- **User Offline:** Notification stored and displayed on next login.- **Delivery Failure:** System logs the failed notification.
User Roles	Customer, Driver, Admin



2.2.15. Account registration function (extension option)

Field	Description
Function ID	F15
Use Case Name	Account Registration
Actor	Public Visitor
Description	Allows new users to create an account by entering personal details and credentials to access the Quickship system.
Precondition	The user is not logged in and has not registered before.
Postcondition	A new user account is created and stored in the system database.
Input	- Full name- Email- Phone number- Password- Role selection (Customer or Driver)
Output	- Registration success message- Option to proceed to login
Main Flow	1. Public visitor selects "Register Account." 2. System displays the registration form. 3. Visitor fills in personal details and creates a password. 4. System validates the input information. 5. If valid, the system creates the new account and displays a success message. 6. User can proceed to the login page.
Alternative Flow	- **Invalid Data:** System prompts user to correct missing or invalid fields.- **Email Already Exists:** System displays "Account already registered."
User Role	Public Visitor



2.3. User Permissions

Function	Customer	Driver	Admin
Login/Logout	✅	✅	✅
Create an order	✅	❌	✅

Function	Customer	Driver	Admin
Single Status Updates	❌	✅	✅
Driver Management	❌	❌	✅
COD Management	✅ (see)	✅ (confirmed)	✅ (compilation)
System Statistics	❌	✅ (cá nhân)	✅ (system-wide)
Language Management	✅	✅	✅

3. Non-functional requirements

Request Group	Detailed Description
Performance	The app must load the page within ≤ 3s; Works smoothly on modern browsers
Bảo mật (Security)	Clear decentralization by user role
Usability	Intuitive interface, support for non-professional users
Scalability	Can be easily transferred to the real system (backend + database)
Đa ngôn ngữ (Localization)	Vietnamese and English language support via LanguageContext
Bảo trì (Maintainability)	The code is organized in a Monolithic Layered model, which is easy to expand and update

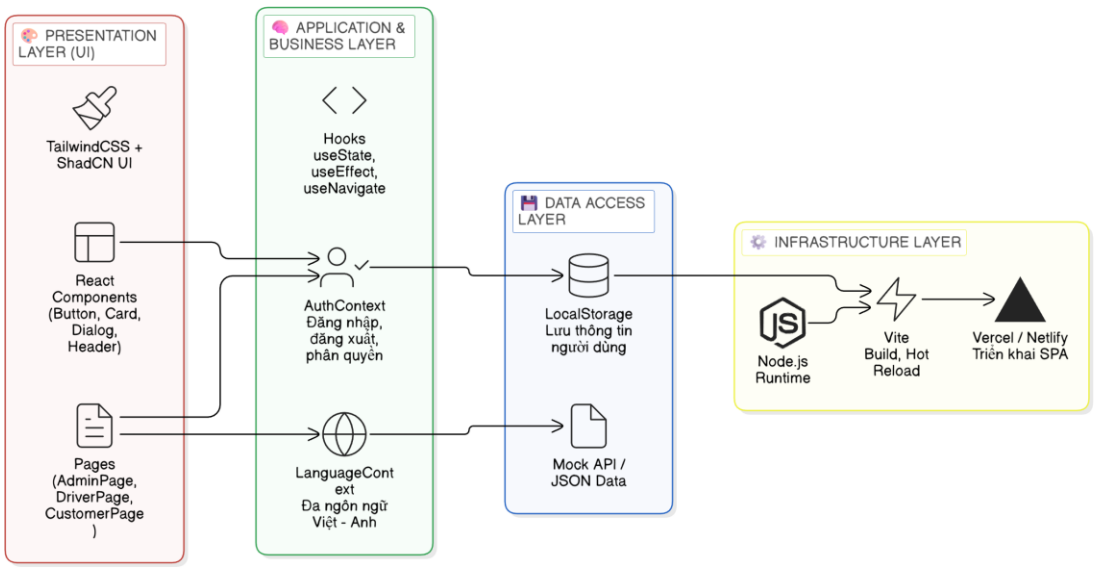
3. ARCHITECTURAL DESIGN & IMPLEMENTATION

3.1 . System Architecture (Kiến trúc hệ thống)

The **Quickship System** adopts a **Monolithic Layered Architecture**, consisting of **4 main layers**.
Each layer is implemented using specific technologies to ensure modularity and maintainability.

Layer	Responsibility	Technology / Implementation
1 **Presentation Layer**	Handles UI rendering and user interaction	React + TypeScript, TailwindCSS, ShadCN UI
2 **Business / Application Layer**	Core application logic and state management	Context API (AuthContext, LanguageContext), React Router
3 **Data Layer**	Local data persistence and API simulation	LocalStorage, JSON Mock API
4 **Infrastructure Layer**	Build, runtime, and deployment environment	Vite (build tool), Node.js (runtime), Vercel (deployment)

3.1.1 . Layered Architecture Diagram (Sơ đồ kiến trúc phân tầng)



3.1.2 TECHNICAL STACK

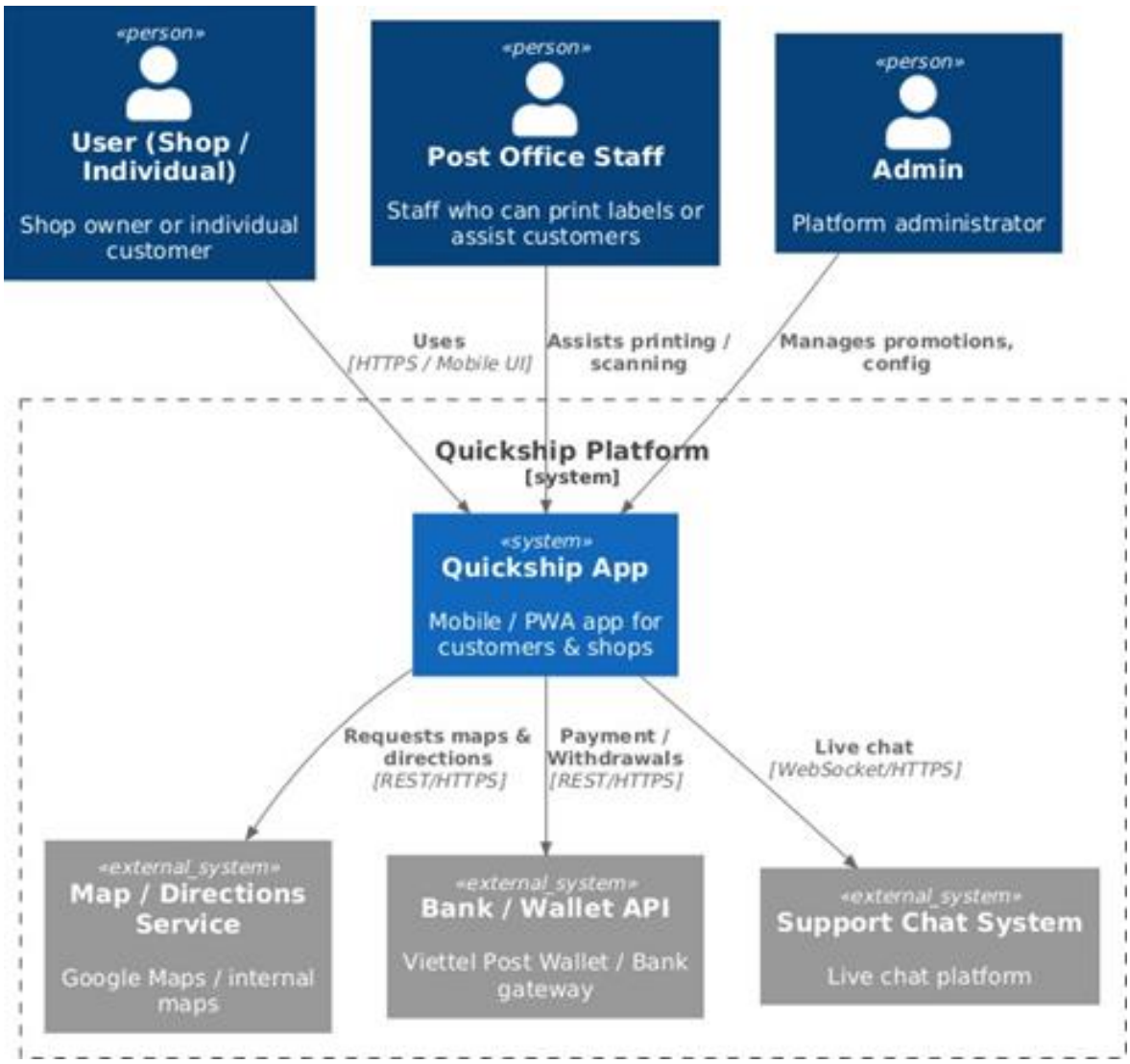
Layer / Category	Technology / Library	Core Functional Description
Primary Language	**TypeScript (TSX)**	Ensures type safety for React code, reducing runtime errors and improving maintainability.
Frontend Framework	**React 18+**	Utilizes a component-based architecture for building a dynamic and high-performance UI.
Build Tool	**Vite**	Provides a lightning-fast development server with Hot Module Replacement (HMR) and optimized ES module builds.
Routing Management	**React Router**	Handles client-side navigation between different views (Home, Login, Admin, Driver, etc.).
UI Framework	**ShadCN UI + Tailwind CSS**	A collection of accessible, reusable UI components styled with utility-first CSS for rapid design customization.
Icons	**Lucide React**	A clean and flexible vector icon library integrated seamlessly into the user interface.
State & Context	**React Context API**	Manages global application states such as authentication (AuthContext) and

Layer / Category	Technology / Library	Core Functional Description
		localization (LanguageContext).
User Management	**LocalStorage + Mock Auth**	Simulates authentication workflows (Login/Logout) and persistent sessions for Admin, Driver, and Customer roles.
Internationalization (i18n)	**Custom LanguageContext**	Supports dynamic bilingual (Vietnamese – English) switching across the entire interface.
Core Components	**Header, BottomNav, Sidebar, Button, Card, Dialog, Table**	A library of modular UI elements reused throughout the application for consistency.
Data Visualization	**Recharts / Custom Charts**	Visualizes key performance indicators (KPIs) such as order statistics, revenue, and delivery performance.
Form Management	**Input / Select / Calendar Components**	Streamlines data entry for order creation, advanced filtering, and user profiles.
Page Architecture	**Admin, Driver, & Customer Modules**	Role-based page routing featuring specific dashboards for each user type (Order Tracking, COD, Promotions).

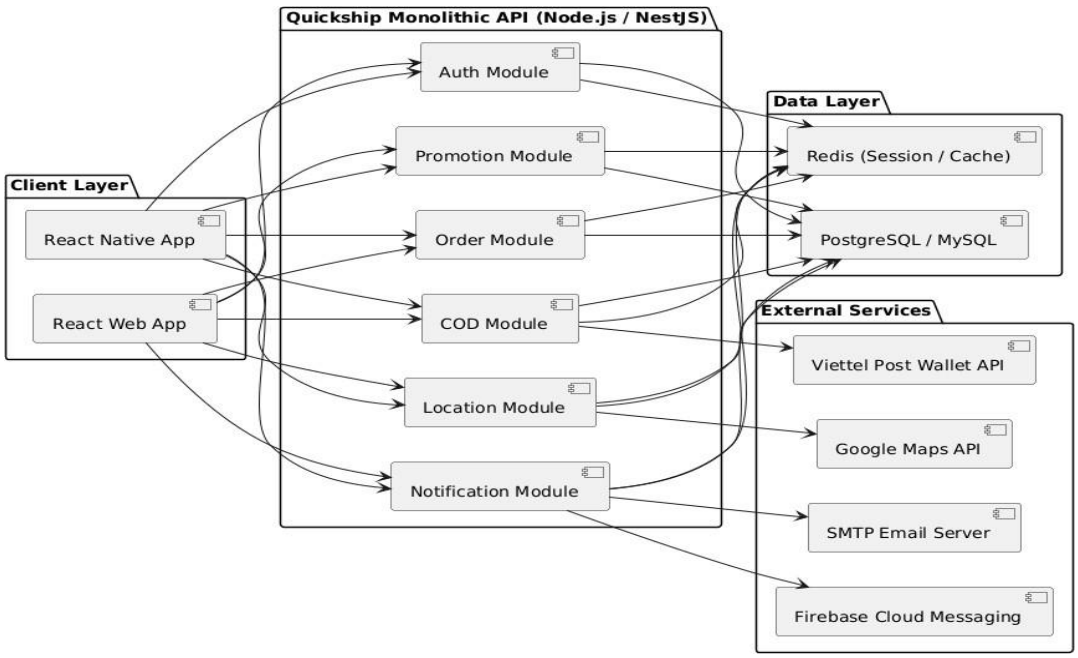
Layer / Category	Technology / Library	Core Functional Description
Development Env	**Node.js + NPM + VS Code**	The standard ecosystem for running the Vite server, managing dependencies, and debugging.
Deployment	**Vercel / Netlify**	Optimized hosting platforms for high-availability Single Page Applications (SPA).
Quality Control	**TypeScript Compiler (tsc)**	Static type checking during the build process to ensure code integrity and prevent bugs.
Project Architecture	**Component-Based + Context Layer**	Modular code organization separating UI, Business Logic (Context), Pages, and Routing.

C4 Model of the Quickship System

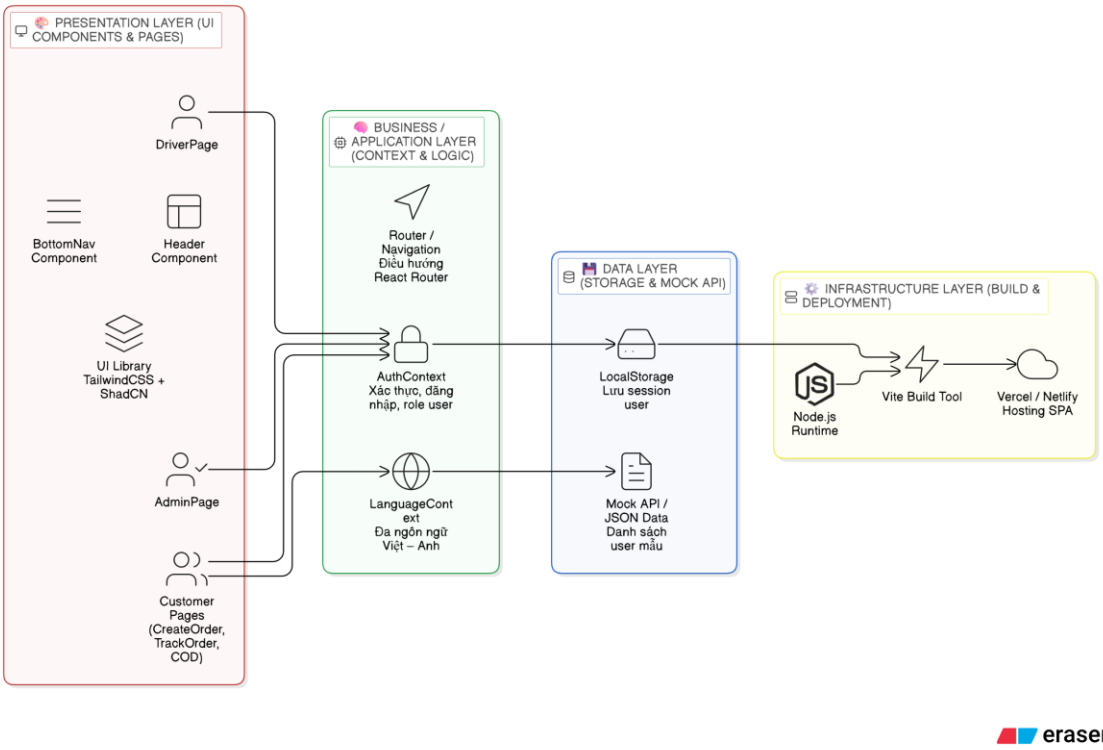
Level 1 – Context Diagram



Level 2 – Container Diagram

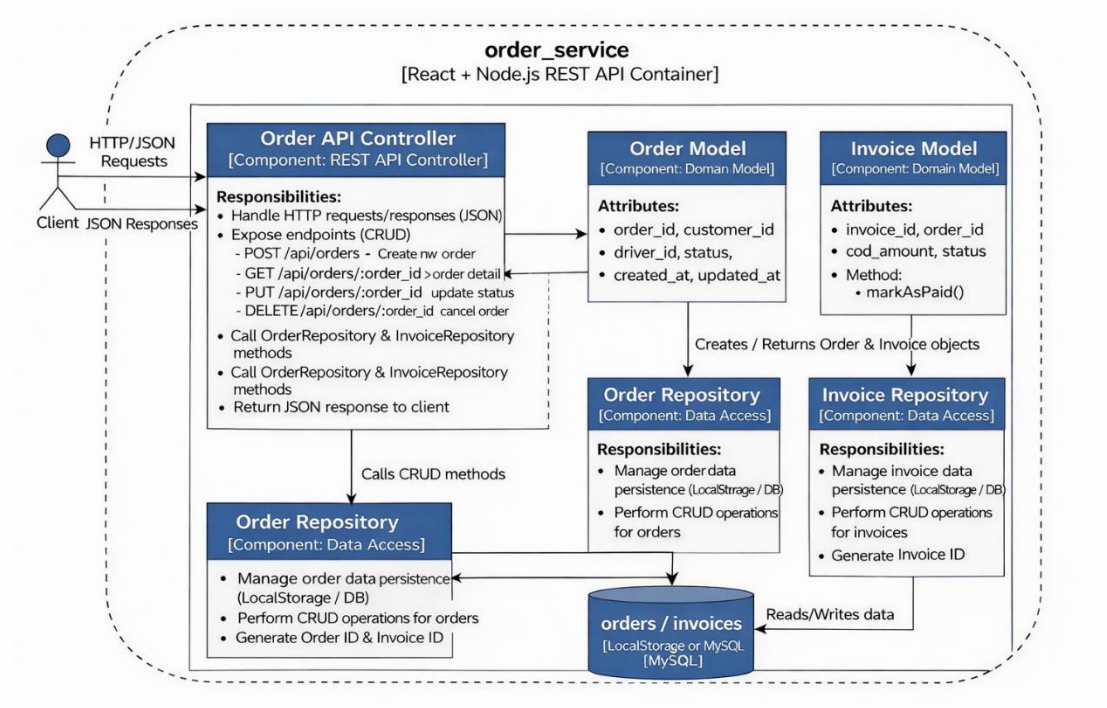


Level 3 – Component Diagram



eraser

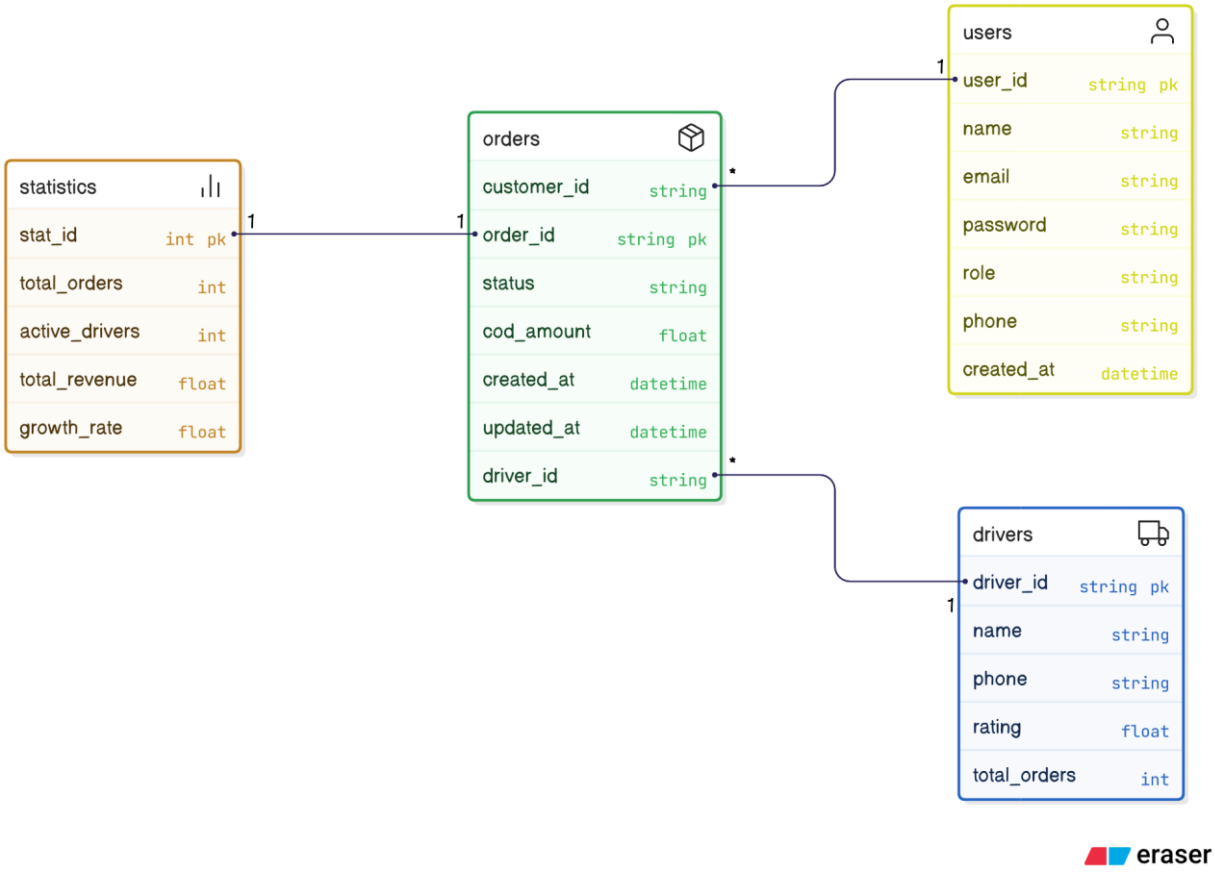
Level 4 – Code level (Implementation View)



The figure illustrates the C4 Code Level for the QuickShip Order Service

3.2 Data Model Design

3.2.1 Logical Data Model



. Physical Design

Bảng users

School Name	**Data types**	**Data types**	**Description**
user_id	VARCHAR(20)	PRIMARY KEY	User Code
name	VARCHAR(100)	NOT NULL	Name
email	VARCHAR(100)	UNIQUE	Email Address
password	VARCHAR(255)	NOT NULL	Encryption Password
role	ENUM('admin','driver','customer')	NOT NULL	User roles
phone	VARCHAR(20)		Phone Number
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Creation Date

Drivers table

Schools	Kiểu dữ liệu	Ràng buộc	Mô tả
driver_id	VARCHAR(20)	PRIMARY KEY	Driver Code
name	VARCHAR(100)	NOT NULL	Driver's Name
phone	VARCHAR(20)	NOT NULL	Driver's phone number
rating	DECIMAL(2,1)	DEFAULT 5.0	Average Rating
total_orders	INT	DEFAULT 0	Number of Orders Delivered

Bảng orders

Trường	Data types	Binding	Description
order_id	VARCHAR(20)	PRIMARY KEY	Bill of Lading Code
customer_id	VARCHAR(20)	FOREIGN KEY → users(user_id)	Booker
driver_id	VARCHAR(20)	FOREIGN KEY → drivers(driver_id)	Driver in charge
status	ENUM('pending','delivering','delivered','cancelled')	DEFAULT 'pending'	Status
cod_amount	DECIMAL(12,2)	DEFAULT 0	Collected money on behalf of
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Creation Date
updated_at	DATETIME	ON UPDATE CURRENT_TIMESTAMP	Last Updated

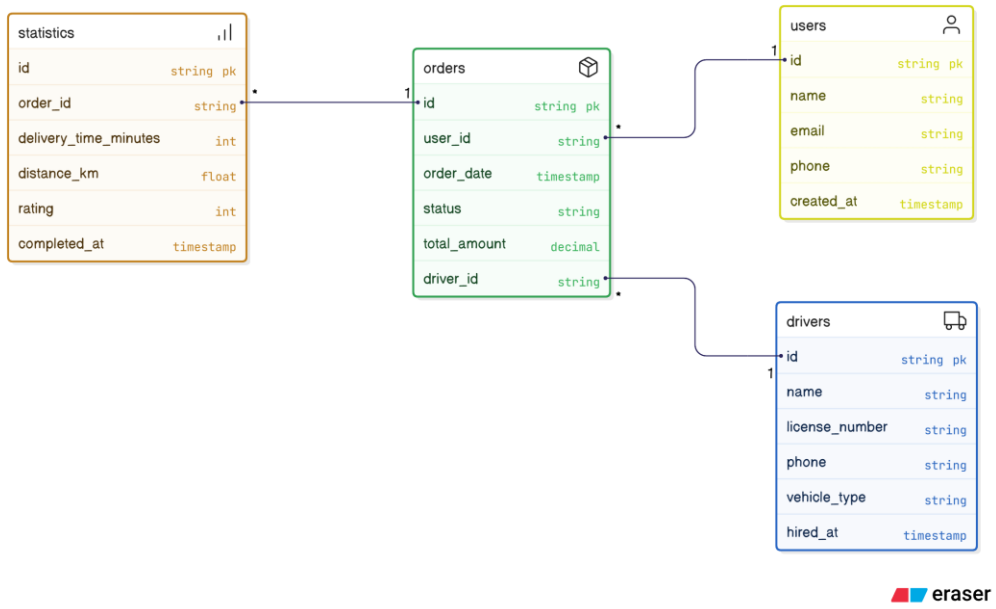
Bảng statistics

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
stat_id	INT	PRIMARY KEY AUTO_INCREMENT	Stat ID
total_orders	INT		Total Orders
active_drivers	INT		Number of active drivers
total_revenue	DECIMAL(12,2)		Total Revenue
growth_rate	DECIMAL(5,2)		Growth rate

. Ràng buộc toàn vẹn dữ liệu

Loại ràng buộc	Mô tả
Primary Key (PK)	Each table has a unique primary key (user_id, order_id, driver_id, stat_id).
Foreign Key (FK)	customer_id → users.user_id ; driver_id → drivers.driver_id.
Unique	The email in Users is unique.
Default / Check	role only receives 3 valid values; status only accepts predefined statuses.
Cascade	When deleting a user, their orders are also deleted (ON DELETE CASCADE).

. Main relationships between tables



A **user (customer)** can create **multiple orders**

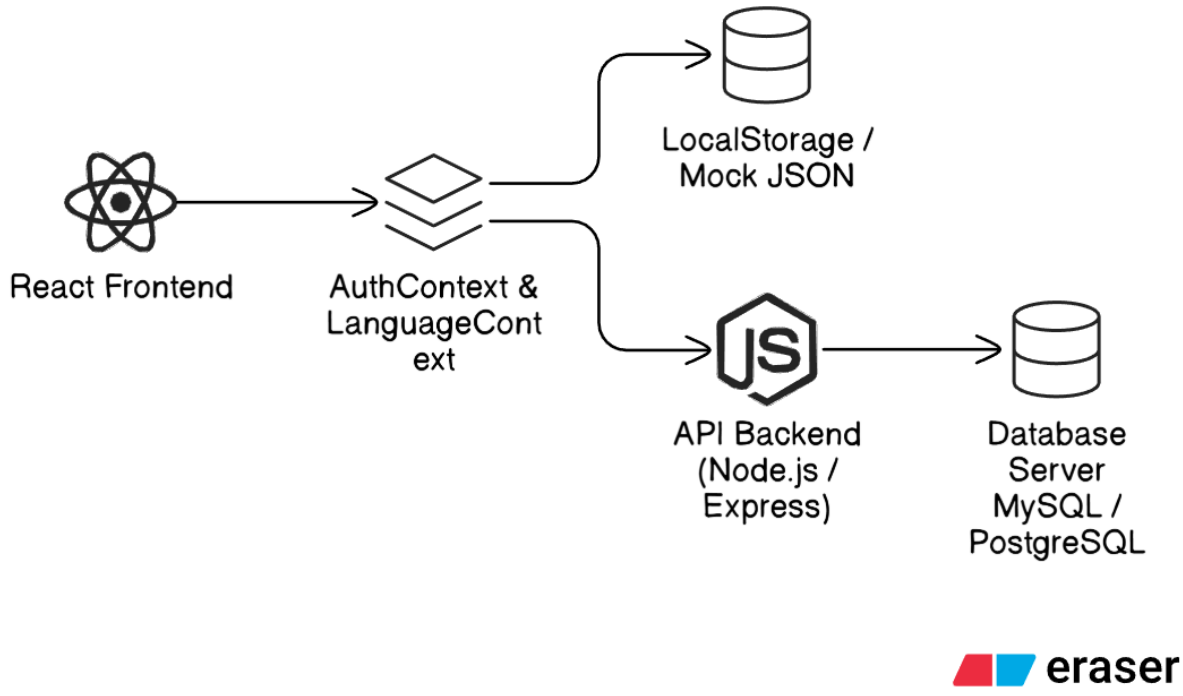
One **driver** can be in charge **of multiple orders**

Statistics table aggregates data from multiple **orders**

. Practical Expansion Proposal

Ingredients	**Solution**	**Objectives**
Database Server	PostgreSQL hoặc MySQL	Stores real data instead of Mock JSON
ORM	Prisma hoặc TypeORM	Secure query management & data mapping
API Layer	Node.js (Express / NestJS)	RESTful communication between the frontend and the database
Authentication	JWT + Refresh Token	User security and decentralization
Backup / Log	Supabase hoặc Railway	Periodic log storage & backups

. Overall data architecture diagram



3.3 Core Components (Thành phần chính của hệ thống)

Component	Responsibility	File / Module
Header	Displays app title and navigation actions	Header.tsx
BottomNav	Quick access navigation bar	BottomNav.tsx
AuthContext	Authentication logic, login/logout flow	context/AuthContext.tsx
LanguageContext	Manages language switching (EN/VI)	context/LanguageContext.tsx
Router	Manages navigation between pages	router.tsx
AdminPage	Dashboard, statistics, driver/order management	pages/AdminPage.tsx
DriverPage	Displays driver-specific orders and COD data	pages/DriverPage.tsx
CustomerPage	Order creation, tracking, promotions	pages/CreateOrderPage.tsx / pages/TrackOrderPage.tsx

3.4 Directory Structure (Cấu trúc thư mục dự án)

```
quickship/
├── src/
│   ├── app/
│   │   ├── components/
│   │   │   ├── Header.tsx
│   │   │   └── BottomNav.tsx
│   │   ├── context/
│   │   │   ├── AuthContext.tsx
│   │   │   └── LanguageContext.tsx
│   │   ├── pages/
│   │   │   ├── AdminPage.tsx
│   │   │   ├── DriverPage.tsx
│   │   │   ├── CreateOrderPage.tsx
│   │   │   ├── TrackOrderPage.tsx
│   │   │   ├── CODManagementPage.tsx
│   │   │   └── LoginPage.tsx
```

```
│   │   └── router.tsx
│   └── assets/
└── vite.config.ts
```

3.5. Code Snippets (Đoạn mã minh họa)

3.5.1. Authentication Logic (AuthContext.tsx)

```
const login = async (email: string, password: string, role: UserRole): Promise<boolean> => {
  await new Promise(r => setTimeout(r, 800));
  const mockUser = MOCK_USERS[role];
  if (mockUser && mockUser.email === email && mockUser.password === password) {
    setUser(mockUser.user);
    localStorage.setItem("viettelpost_user", JSON.stringify(mockUser.user));
    return true;
  }
  return false;
};
```

3.5.2. Create Order Logic (CreateOrderPage.tsx)

```
const handleCreateOrder = () => {
  const newOrder = {
    id: "ORD" + Date.now(),
    customer: user.name,
    cod: codAmount,
    status: "pending"
  };
  const orders = JSON.parse(localStorage.getItem("orders") || "[]");
  orders.push(newOrder);
  localStorage.setItem("orders", JSON.stringify(orders));
  alert("Order created successfully!");
};
```

3.5.3. COD Summary View (CODManagementPage.tsx)

```
const totalCOD = orders.reduce((sum, o) => sum + o.codAmount, 0);
return (
  <div>
    <h3>Total COD Collected: {totalCOD}đ</h3>
    {orders.map(o => (
```

```
    <p key={o.id}>{o.id}: {o.codAmount}đ</p>
   ))}
  </div>
);
```

3.5.4. Statistics Dashboard (AdminPage.tsx)

```
<TrendingUp className="text-red-600" />
<p>Weekly Performance: +25% vs last week</p>
<div className="w-full bg-gray-200 rounded-full h-2">
 <div className="bg-red-600 h-2 rounded-full" style={{ width: "80%" }}></div>
</div>
```

4. TESTING & VERIFICATION

4.1 Testing Approach

The QuickShip system was tested using **functional black-box testing** to verify that all implemented features operate according to the defined requirements.

Testing environment:

- Browser: Google Chrome
- Data Source: LocalStorage (Mock Data)
- Development Server: Vite

4.2 Functional Testing Summary

1 Authentication

Valid credentials → Redirected to correct dashboard ☑

Invalid credentials → Error message displayed ☑

2 Order Creation

Valid input → Order created with unique ID ☑

Missing fields → Validation error triggered ☑

3 Order Tracking

Valid tracking code → Correct status displayed ☑

Invalid code → "Shipment not found" message shown ☑

4 Driver Management (Admin)

Add / Edit / Delete driver → System updates correctly ☑

5 COD Management

COD total is calculated using:

```
const totalCOD = orders.reduce((sum, o) => sum + o.codAmount, 0);
```

Multiple orders → Correct total displayed ☑

No COD orders → Returns 0 ☑

6 Language Switching

Switching EN ↔ VI updates UI instantly ☑

4.3 Performance & Security

- Page load time: < 3 seconds
- Role-based access control enforced
- Session stored via LocalStorage

Limitation: JWT authentication not yet implemented.

4.4 Conclusion

All major modules function correctly without critical errors. The system satisfies core functional requirements and is stable for prototype deployment.